

自前 printf — 数を文字に変える

今日のゴール

libc の `printf` を使わずに、自分で `print_str` / `print_int` / `printf1` を書く。自作コンパイラでコンパイルした自前の出力ライブラリで、libc なしのプログラムを動かす。

1 この回の位置づけ

- ・ ランタイム発展シリーズの第 2 回（選択制）。前提は R1（`sys_write` が使える状態）
- ・ 書くのはすべて **Core** プロファイルの **C**。自分のコンパイラでコンパイルする
- ・ コマ 11 で「`printf` 自体は実装しない」と言った、その中身を作る

2 可変長引数は定義できない — どう設計するか

本物の `printf` は可変長引数関数である。

```
int printf(char *fmt, ...);
```

しかし Core プロファイルでは、`...` は外部関数の宣言だけに許され、自分で定義することはできない（`language_spec.md` の「関数」節）。可変長引数の実装には、スタック上の引数を型情報なしに読み出す仕組み（`va_arg` 相当）が必要で、コンパイラ側の対応も要るからである。

そこで、引数が固定の関数を組み合わせる設計にする。

関数	役割
<code>print_char(int c)</code>	1 文字書く
<code>print_str(char *s)</code>	NUL 終端文字列を書く
<code>print_int(int n)</code>	整数を 10 進で書く
<code>printf1(char *fmt, int iarg, char *sarg)</code>	書式つき（引数 1 個ぶん）

`printf1` は「整数の引数」と「文字列の引数」を両方受け取り、書式指定子に応じてどちらかを使う。使わない方には 0 を渡す。

```
printf1("n=%d\n", 42, 0);
printf1("s=%s\n", 0, "hello");
```

不格好に見えるが、言語の制約の中で目的を達するのは実装の常である。

3 数を文字に変えるということ

`print_int` がこの回を中心である。整数 1234 を画面に出すには、文字コードの列 '1' '2' '3' '4' を作る必要がある。

`n % 10` で下の桁が取れるが、これは最後に表示したい桁である。そこで、バッファの末尾から手前へ詰めていく。

`buf[16]` を用意し、`i = 15` から始める

```
n = 1234:
  buf[15] = 0      ← NUL 終端
  buf[14] = '4'    n % 10 = 4、n = 123
  buf[13] = '3'    n % 10 = 3、n = 12
  buf[12] = '2'    n % 10 = 2、n = 1
  buf[11] = '1'    n % 10 = 1、n = 0 → ループ終了
```

`print_str(&buf[11])` ← 詰め始めた位置から出力

数字を文字にするのは '0' + 桁である ('0' は 48)。コマ 11 で文字が小さな整数であることを見たが、ここではその逆向きの変換をしている。

重要

`n == 0` の場合を忘れない。while (`n > 0`) のループは 1 度も回らないので、何も出力されない。0 は特別扱いして '0' を 1 文字置く。

負数を忘れない。先に符号を覚えて `n = -n` してから桁を作り、最後に '-' (45) を先頭に足す。

4 書式の解釈

`printf1` は `fmt` を 1 文字ずつ見て、`'%'` (37) が来たら次の 1 文字で分岐する。

指定子	文字コード	動作
<code>%d</code>	100	<code>print_int(iarg)</code>
<code>%s</code>	115	<code>print_str(sarg)</code>
<code>%c</code>	99	<code>print_char(iarg)</code>
<code>%%</code>	37	<code>'%'</code> を 1 文字出す

Core プロファイルには文字リテラルがあるので `fmt[i] == '%'` と書けるが、参照実装では数値で書いてある (どちらでもよい)。

これが、コマ 11 で `printf("x=%d\n", 42)` を呼んだときに `libc` の中で起きていたことである。書式文字列は、実行時に解釈されている。

5 実装

Step	実装対象	内容
1	<code>my_strlen</code> / <code>print_char</code> / <code>print_str</code>	文字列の基本
2	<code>print_int</code>	10 進変換 (この回の山場)
3	<code>printf1</code>	書式の走査と分岐

`sys_write` と `_start` は R1 の `syscall.s` (配布済み) から来る。

手で動かす場合

```
python3 ../../../../final/mycc.py myprintf.c > impl.s
python3 ../../../../final/mycc.py tests/fmt.c > main.s
riscv64-linux-gnu-gcc -nostdlib -static main.s impl.s syscall.s -o prog
qemu-riscv64 ./prog
```

6 テスト

```
python3 check.py
```

`tests/*.c` を自作コンパイラでコンパイルし、`myprintf.c` と `syscall.s` とリンクして実行し、標準出力と終了コードを確認する。**libc** は一切使わない。

7 発展課題

1. **%x (16 進)** を足す: 10 以上の桁を 'a' ~ 'f' にする分岐が必要になる
2. 桁揃え: `%5d` のように幅を指定できるようにする (先に桁数を数えて空白を出す)
3. 引数を増やす: `printf2(fmt, i1, i2)` を作る。なぜ本物の `printf` が可変長引数でなければ困るのかを実感する
4. **バッファリング**: いまは 1 文字ごとに `sys_write` を呼んでいる (システムコールは遅い)。出力を配列にため、いっぱいになるか改行が来たときにまとめて書く方式にして、`sys_write` の呼び出し回数を比べる
5. **print_int** の限界: `-2147483648` (int の最小値) を渡すとどうなるか。 `n = -n` が正しく動かない理由を考える

補足

コラム: libc も C で書かれている。いま書いた `print_int` は、glibc の `printf` の中にある 10 進変換とほとんど同じことをしている。違いは、バッファリング、ロケール、浮動小数点、スレッド安全性…といった「実用のための膨大な付加物」である。核心の「数を文字に変える」部分は、いま自分で書いたものと変わらない。

8 次回予告

出力ができるようになった。R3 では**メモリの割り当て**を自分で行う。コマ 13 で「実装しない」とした `malloc` の中身を作る。